

Feedback on Prototype: Non-technical audience

For the gathered feedback from non-technical users, I was able to observe key findings, I followed the plan that was previously described. I demonstrated how the prototype worked; its purpose and current key working features. After I had encouraged them to try it out themselves and work around with the prototype. A lot of positive results were gotten from this target audience and also some key findings discovered. Below is a summary of the questions asked:

1. How would you rate the layout of the website?
2. How did the overall aesthetics of the website appeal to you?
3. How were the feedback error messages provided? were they confusing and vague or easy to understand
4. I also asked them about the quiz how easy it was interacting with the main features, including navigation, account registration, appointment booking and accessing accounts.
5. I concluded the form asking them what features they would like to see in the next iteration of the website.

I finally had conversations with most of them and got to know what they liked the most, and these were most of their Responses:

"I really like how the website is not cluttered, everything is understandable and minimal"

"The best section, was towards the end of the carbon footprint calculation, I was informed about my carbon footprint score in an interactive way, with the graphs and a little advice by the side"

Other data responses drawn from the form include:

1. APPOINTMENT REGISTRATION:

About 25% of the users found it difficult registering their accounts. This was mainly a systems bug that would time out the connection if left idle for over 30 minutes, I advise that to address this issue, a configured TLS handshake could be integrated as this also ties closely with browsers.

CROSS ORIGIN POLICIES. See <https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/CORS> for more information on CORS as a technical developer. This often led me to refresh the VSCode Live Server Extension, as nothing was wrong with the Express Server. This was not a major issue as it was quickly addressed. The other 75% of the users found it very easy to use.

2. CARBON FOOTPRINT RESULTS:

Question1: "Were you able to see your results after you calculated your carbon footprint"

Question2: "How would you experience calculating your carbon footprint"

An average of 5 star rating was given to Question1, as I made sure the final total of the user was clearly displayed and easy for them understand supporting it with a graph analysis. This was also the same result for question 2 as all users enjoyed the feature. However, I decided that to improve it, it would be greatly beneficial to allow users to download their report and save it to their device, this can be looked forward to in the next iteration of the prototype.

3. FEATURE RECOMMENDATIONS:

Question: "What would you change about the website or what feature would you want to see next"

While most of them were satisfied, I had gotten two key responses:

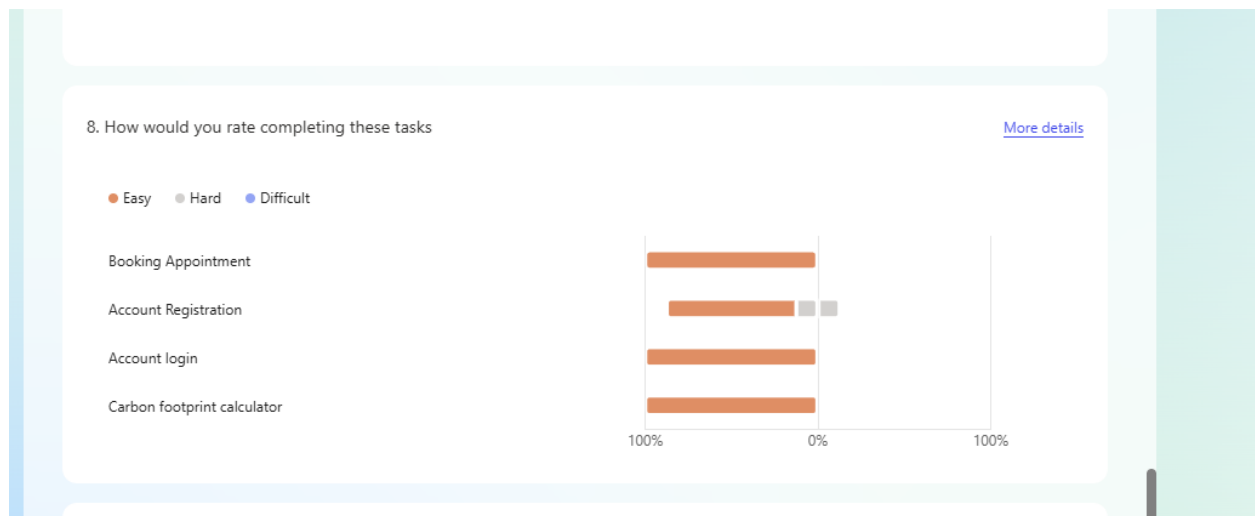
"Maybe, make it so the password has requirements"

"I wouldn't change anything in particular however we could probably try and go slightly into more detail with all the information provided"

This is a key insight as it points to a minute hole in User Experience, which is a key factor in any digital space. To address this, the HTML and EJS files responsible for rendering this content should have password requirements stated by the side of the form element.

Also, regarding the second response, I figured that a dedicated section to products and services offered by Rolsa Technologies would suffice, as in this prototype version, I had just made a summary of the services to provide without explicitly explaining them.

The website received an overall average rating of 4.75 for User Experience which signifies a great prototype to build upon, but needs minor adjustments to make it a 5.00 rating overall. Please see the Rolsa Technologies.csv file for more detailed information



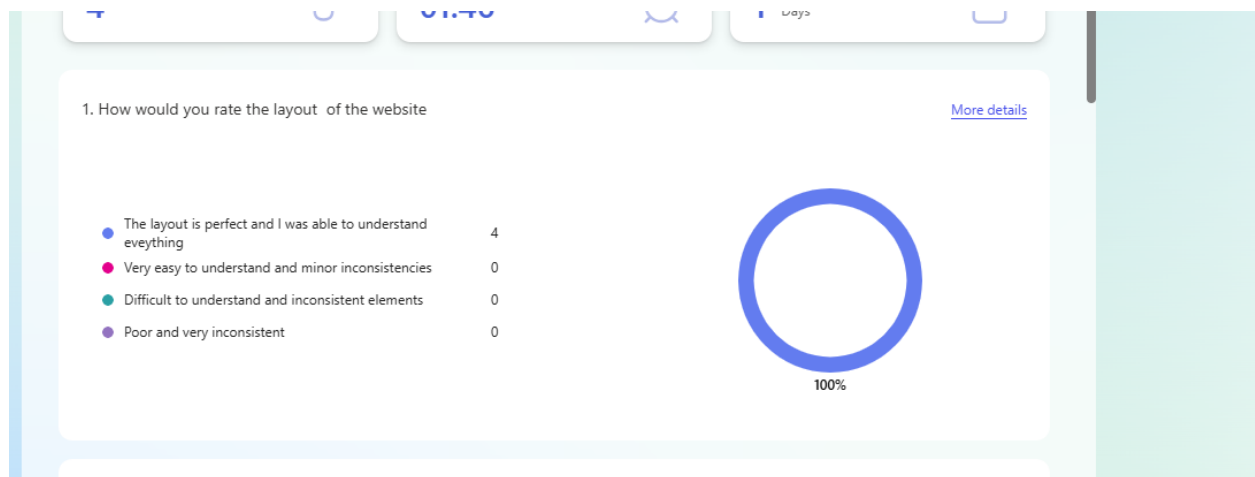
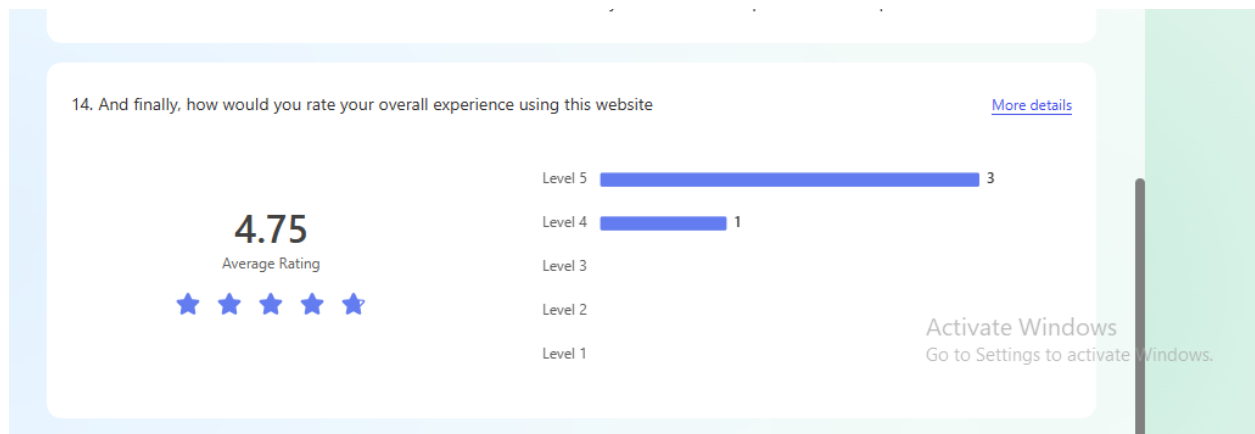
Other observations among all users include the following:

- They were easily able to navigate the user interface
- The technical audience were more engaged into the source code as I talked them through my design choices.
- Users who only used keyboards were able to easily navigate the website without issues.

13. What would you change about the website or what feature do you want to see next ✕

2 Responses

ID ↑	Name	Responses
1	anonymous	Maybe make it so the password has requirements
2	anonymous	I wouldn't change anything in particular however we could probably try and go slightly more into detail with all the information provided.



As regards the technical audience, I had made sure they received a separate kind of form, tailored to them. I had made a separate form for them. I first started by explaining the System Architecture to them, including the backend server, database schema and reasons for change. I had asked an experienced backend developer what they would approach to find a similar but more robust solution than mine. They had advised that it would have been better to make use of two different databases, a NoSQL one ie Firebase & Supabase together with the MySQL used. So, I can explicitly control user

authentication locally on my machine, while the cloud database mainly stores miscellaneous data like extra bookings, managing concurrent possible states when it comes to scaling and more. This is a reasonable advice and is clearly backed up as this was also one of the issues I tried addressing as stated in my Changelog.pdf file, by making a dedicated section in the MySQL database, to handle miscellaneous data and state that can easily be retrieved.

These were the main questions asked to this subset of audience, and this were their responses on the Technical Form:

1. How scalable is the current prototype
2. How would you rate the integration process with other systems (Monolith or microservices)
3. What other possible test suites could be added?

All the above questions received extremely positive feedback as they answered there were little to no issues with integration and scalability, the performance of the system was blazingly fast, otherwise "Excellent, page load times very quick". No other test suites were recommended from the audience I was able to interact with but there should have been more testing on the code, as compared the overall system testing, I had done in "v1/tests/endpoints.tests.js". Example:

1. Testing the hashing function: Is a user's password correctly hashed before saving to the database? It did hash their passwords, but for future developers it would be easy for them to just navigate the test for that particular function instead of re-writing it.

2. More testing on the VSCode live server has raised minor issues, it would be better to move everything onto the same server, ie All pages are on rendered by the Express Server.

Other data responses drawn from the questionnaires include:

1. INDUSTRY STANDARD PRACTICES

Question: "How does the source code go well with standard development practices?"

All agreed that the code practices I used were very good as the code was self-descriptive, the code was well modularized avoid over abstraction and constant manipulation of global variables, meaningful comments. However, I would say that more documentation on the setup process would do, as it is possible a newly hired developer into the company might not really know how to set up the environment on their device whether they are on a Linux based Operating system, or implemented Docker to help with the configuration process, also including debugging issues. I had a developer who advised that when I'm about to deploy the prototype, I should avoid logging activities into the sever terminal but rather, into a text file, that way, it is much cleaner and safer, and well documented.

2. SERVER RESPONSE

Question: "The current prototype makes use of SSR, and server response time is < 12ms, what do you think can be improved?"

These was a key response that was received:

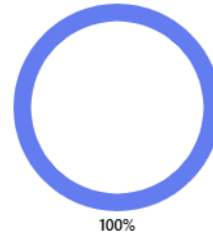
"Maybe using state variable or using a framework, i.e. Svelte or Next.js could improve the speed of the prototype"

After considering and thinking about their response, I came to the conclusion that including a framework is a good idea. But the frameworks provided by this user are frameworks that are met for large scale Front-end applications and not mainly for SSR (Server Side Rendering), but instead, integrating HTMX, which is also a framework that allows for extremely quick page load times could be considered, including ASTRO which is a framework that is built of SSR concept. However, the end product requested by the company did not need extreme Reactivity like how popular apps like Twitter, Instagram would need. This can in fact, reduce page load speeds, and cause significant bounce rates from the website. Svelte, however, is a choice that can be considered, but for a product that does not require intense reactivity, but just regular API calls, HTMX and ASTRO could greatly help. But in efforts made, I had ensured that the Stylesheets were minified by a compiler ie make it quicker for the website to understand and read and made sure that the server rendered EJS templates instead of regular HTML. And also using CDN instead of local npm packages ie flatpickr, Chartjs. The system architecture was thoroughly planned out in the beginning of the project and has reasons for specific choices. More details can be found in the Technical_Response.csv file. Here are some pictures:

1. How scalable is this existing prototype?

[More details](#)

● Very scalable with no issues	4
● Fairly scalable with minor issues	0
● Poor	0



2. How would you rate this prototype integrating with other systems (monolith or microservices)

[More details](#)

● Little to no issues with integration	4
● The prototype will still need some refactoring before integration	0
● The prototype will create several integration issues	0



8. Database security: I made use of parameterized queries to avoid SQL Injection. How would you rate this approach?

[More details](#)

4.25

Average Rating



Level 5  1

Level 4  3

Level 3

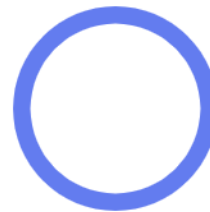
Level 2

Level 1

9. How does this meet the modern standards of the web development practices

[More details](#)

- Very good 4
- Good 0
- Fair 0



Activate Windows
Go to Settings to activate Windows.



In summary, both audiences were satisfied with the prototype as it received an average of 4.75 ratings. While there were only a handful of users who had used a similar software similar to this, most of them after interacting with them said this was their first time, especially knowing about "Carbon footprint". To make sure that the next iteration cycle gets better results, these are a list of future improvements that can be proposed and further investigated by future developers:

1. More testing coverage on individual components of the system.

2. Adding password requirements for users to give a better User Experience
3. Make use of two or more database systems, Cloud storage and local storage, i.e. Supabase/Firebase & MySQL for more data redundancy, scalability and storage.
4. Introducing new frameworks to help with system performance and other metrics i.e. SEO optimization, reduced lagging effects and adaptable system architecture. These frameworks include ASTRO, HTMX, Svelte.
5. Introducing a new backend server built in Golang, if necessary (if more scalability and performance is needed)
6. Using self-contained programs to make sure that other engineers can carry out testing and other operations with minimal setup required, i.e. using Docker or Kubernetes
7. More documentation on the back-end setup as different developers might have a different setup tailored to their development purposes.
8. Incorporating a logging tool for all server activities and backup
9. Integrating cookies for http only cookies as session persistence: i.e. to identify who a user is and logging them out appropriately and more securely.
10. Hiring security experts to test reverse engineering vulnerabilities and network vulnerabilities.
11. Introducing OAuth for future users